Ex 14: Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Topic: Patient Appointment Booking System

V Vijay Prakash
241401128

1. Aim

To explore how a low-code Zoho Creator application can be extended with code: first by writing reusable custom functions in Deluge, and then by calling an external serverless function built on Zoho Catalyst, using the Patient Appointment Booking application as the example.

2. Procedure

Step 1: Identify what low-code does not cover easily — The booking application's logic was reviewed to find tasks that are better written as code: logic reused in several workflows, and processing that is awkward to build with the visual tools, such as text formatting or data cleaning.

Step 2: Write a custom function in Creator — From the Workflow area, the Functions section was used to create a reusable Deluge function that checks whether a doctor, date and time slot is already booked, so that several workflows can call it instead of repeating the same query.

Step 3: Call the function from a workflow — The function was called from the form's validation workflow in place of the inline clash check.

Step 4: Create a Catalyst function — In Zoho Catalyst, a serverless function was written in Node.js that receives booking details and returns a cleaned phone-number check and a formatted confirmation message.

241401128

Step 5: Connect Creator to Catalyst — The Catalyst function's endpoint was called from Deluge with the invokeurl task, authorised through a connection or an OAuth token, as in the Zoho CRM integration of Ex 7.

Step 6: Test and review — Both extensions were tested with sample bookings, and the responses were compared with the expected results.

3. Output

3.1 Architecture of the Pro-Code Extension

Zoho Creator keeps the booking form and its workflow. When a booking is submitted, the workflow calls a Deluge custom function, and the Catalyst function is reached through an HTTPS request sent with invokeurl over a connection. The Catalyst function returns a JSON response to Creator.

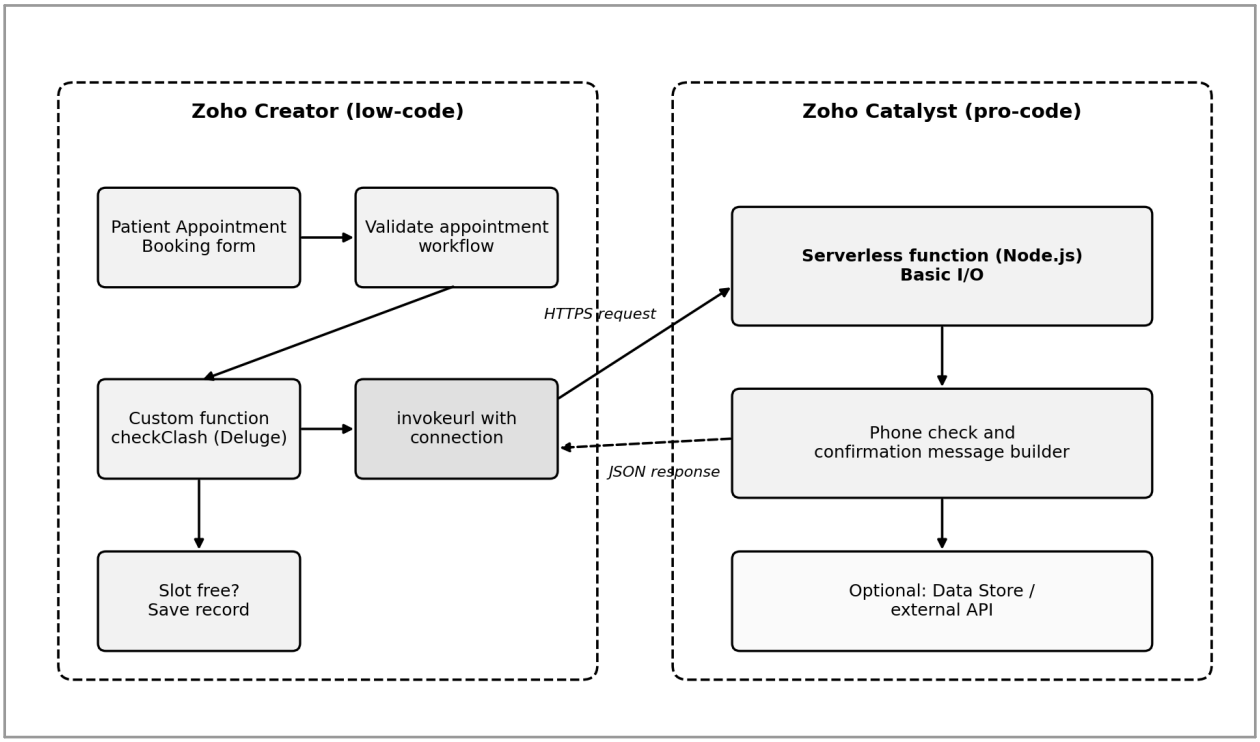


Figure 1: Architecture of the Creator and Catalyst Extension

241401128

3.2 Execution Flow of the Extended Workflow

Figure 2 follows a booking from submission to saving. If the response from Catalyst is not received, the error is logged and the booking continues with a default message, so that a booking is never lost because of the extension.

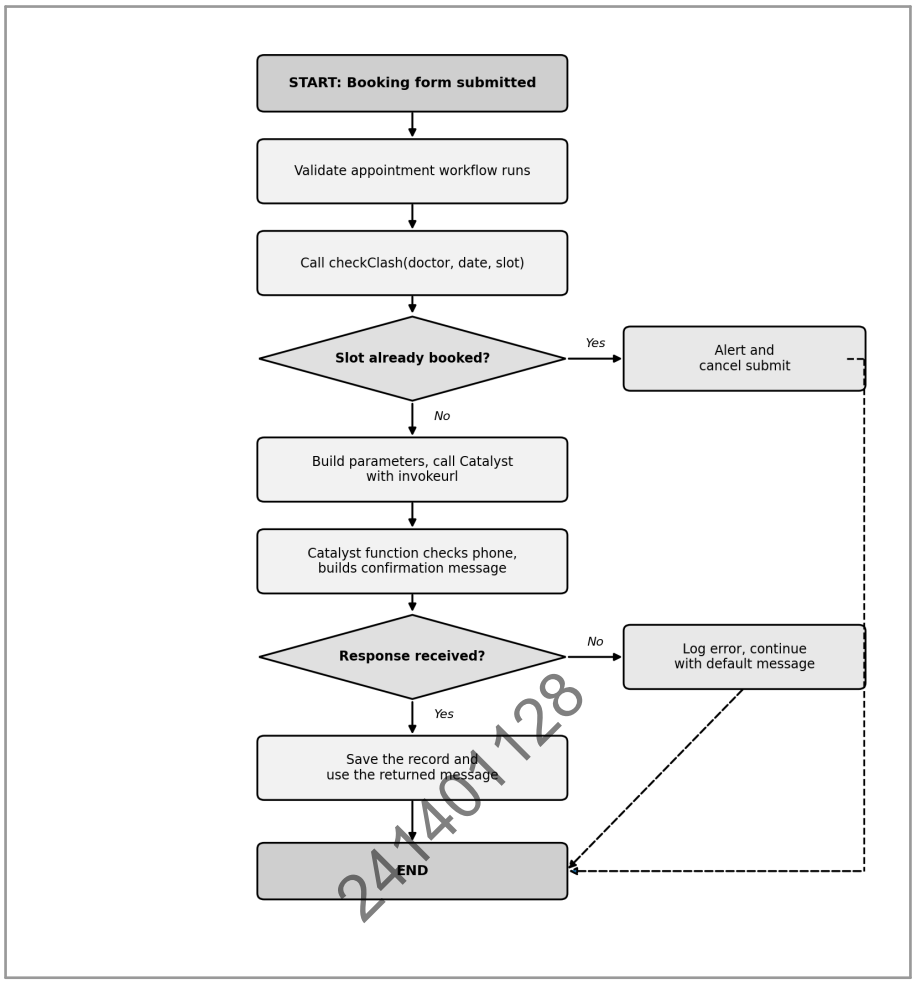


Figure 2: Execution Flow of the Extended Booking Workflow

3.3 Choosing Between Low-Code and Pro-Code

Need	Low-Code Option	Pro-Code Extension
Reuse one rule in many workflows	Repeat the script in each workflow	Custom Deluge function called from each workflow
Heavy or specialised processing	Limited by what Deluge offers	Catalyst serverless function in Node.js, Java or Python
Use of external libraries	Not available in Deluge	Packages included in the Catalyst function
Scheduled or event-driven back-end tasks	Creator schedules and workflows	Catalyst functions, cron and event listeners

Table 1: When a Pro-Code Extension Helps

3.4 Custom Function in Zoho Creator (Deluge)

The function below returns how many bookings already exist for the given doctor, date and time slot:

```
int appointments.checkClash(string dr, date d, string slot)
{
    n = Patient_Appointment_booking[doctor == dr && Appointment_Date1 == d
        && time_slot == slot].count();
    return n;
}
```

It is then used in the validation workflow in place of the inline query:

```
if(thisapp.appointments.checkClash(input.doctor, input.Appointment_Date1,
   input.time_slot) > 0)
{
    alert "This slot is already booked. Please choose another.";
    cancel submit;
}
```

3.5 Serverless Function in Zoho Catalyst (Node.js)

A Catalyst Basic I/O function that checks the phone number and builds the confirmation text:

```
module.exports = (context, basicIO) => {
  const name   = basicIO.getArgument("name");
  const doctor = basicIO.getArgument("doctor");
  const date   = basicIO.getArgument("date");
  const slot   = basicIO.getArgument("slot");
  const phone  = basicIO.getArgument("phone") || "";

  const digits = phone.replace(/\D/g, "");
  const result = {
    valid_phone: digits.length >= 10,
    message: "Dear " + name + ", your appointment with " + doctor +
             " on " + date + " at " + slot + " is confirmed."
  };
  basicIO.write(JSON.stringify(result));
  context.close();
};
```

24140 1128

3.6 Calling the Catalyst Function from Creator

```
params = Map();
params.put("name", input.patient_name.first_name);
params.put("doctor", input.doctor);
params.put("date", input.Appointment_Date1);
params.put("slot", input.time_slot);
params.put("phone", input.Phone);

resp = invokeurl
[
    url: "<CATALYST_FUNCTION_URL>"
    type: GET
    parameters: params
    connection: "<CONNECTION_NAME>"
];
info resp;
```

The function URL and connection name are placeholders for the values of the Catalyst project. Credentials are kept in a connection rather than written in the script, in line with the practice described in Ex 12.

3.7 Test Cases

Test	Input	Expected Result	Observed Result
Custom function, slot free	Doctor, date and slot with no booking	Returns 0; booking is saved	
Custom function, slot taken	Same doctor, date and slot as an existing booking	Returns 1 or more; alert shown and submission cancelled	
Catalyst call, valid phone	10-digit phone number	valid_phone is true; confirmation message returned	
Catalyst call, short phone	Fewer than 10 digits	valid_phone is false	

Table 2: Test Cases for the Pro-Code Extensions

4. Result

A pro-code extension approach was explored for the Patient Appointment Booking application. A reusable custom function was designed to centralise the duplicate-booking check, and a Zoho Catalyst serverless function was designed to handle processing that goes beyond what Deluge offers, called from Creator with invokeurl. The code samples are illustrative and need to be tested in the live Creator and Catalyst accounts, where plan limits may apply.

241401128